COMPACT AI HACKATHON · BUILD NEXT 2026

Project Portfolio

PatteVaidya

On-device crop disease detection for Indian farmers

A 4.36 MB model that names a leaf disease from one photo and tells the farmer what to do about it — fully offline, with no account and no data leaving the phone.

Team	Krishi Lens
College / Institute	IITG
Members	**Harshit Dwivedi** — dataset curation, model training and ONNX export, Leaf Guard, advisory engine, Gradio app, profiling, documentation, training kit and portfolio **Mahak Jaiswal** — design and research, UI/UX, pitch and demo, problem research **Nimish Garg** — front-end engineering, application interface and camera flow
Contact	dubeybittu026@gmail.com
Repository	github.com/harshitd842/hackathon
Submission date	October 2026

1. At a glance

Everything below is measured on the model that ships in this repository, not estimated.

What it is	An offline crop-disease detector with a treatment advisory, built as a local Gradio application
Model	MobileNetV3-Small (ImageNet weights, frozen) used as a feature extractor, plus a trained linear head
Model file	crop_disease.onnx — 4.36 MB, fp32
Inference	0.7 ms per image, 15 MB of RAM, one CPU core, no GPU
Coverage	38 classes across 14 crops: 12 healthy classes and 26 disease or pest classes, covering 20 distinct disease names
Measured accuracy	90.1% on 1,140 held-out images the model never saw in training
Data leaving the device	0 bytes — no photo, no location, no account
Stack	Python 3.12, PyTorch, ONNX Runtime, Gradio
Licence position	Our own trained head; base weights from torchvision; dataset licences listed in Section 12

2. Problem statement

Every season, Indian farmers lose a large share of their crop to pests and diseases. Most of them have no quick, affordable way to find out what is wrong with their plants. Diagnosis today depends on agricultural extension officers who are few and far between, or on guesswork — which often leads to the wrong pesticide, over-spraying, higher cost and lower income.

A farmer in a remote village may own a smartphone but have weak or no internet, so cloud-based AI tools are of little use. That is the gap this project targets: a model small enough to live on the phone, fast enough to answer in a moment, and honest enough to say when it does not know.

3. Why it matters

The scale of the loss is documented in published Indian agricultural research:

- Pre-harvest crop loss due to pests and diseases in India is estimated at around 20%.
- Crop loss due to insect pests is estimated at about 25% in rice and maize, 15% in pulses, and up to 50% in cotton.
- Plant-parasitic nematodes alone cause 21.3% crop loss, worth roughly ₹10,200 crore (about US$1.58 billion) every year.
- In a survey of Indian tomato farmers, average yield loss from pests and diseases was about 11 tonnes per hectare.
- Around 86% of Indian farmers are small or marginal landholders (Agriculture Census 2015–16), and can least afford a failed season.

A fast, offline diagnosis directly reduces wrong pesticide use, cuts input cost, and protects yield — with no dependence on connectivity or on an officer's visit.

4. What we built

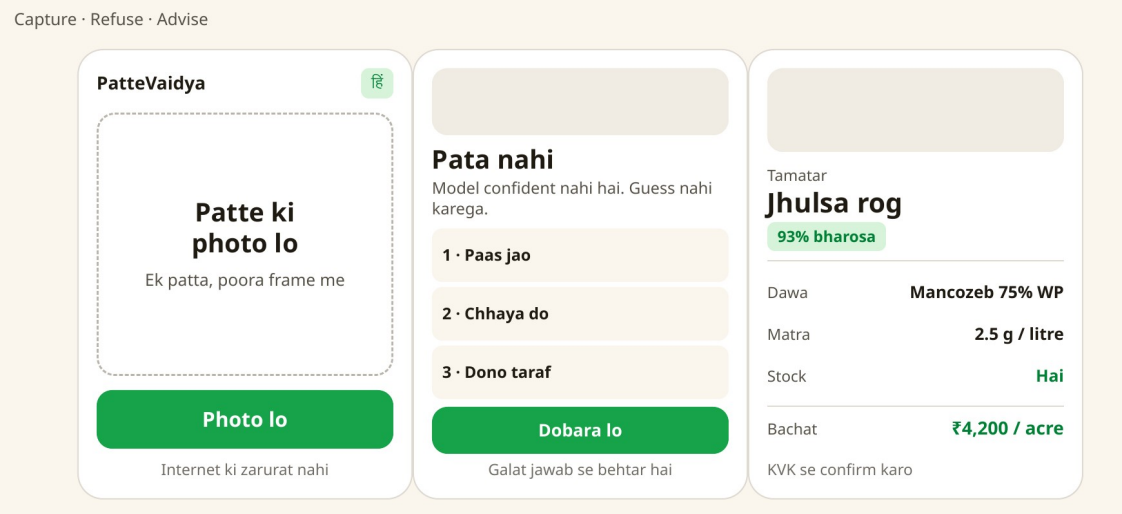
This is not a proposal. The pipeline, the model, the guard, the advisory and the demo all run today.

The five parts

- **Leaf Guard —** refuses the photo when it is not a supported crop or when the model is not confident, instead of guessing.
- **Leaf segmentation —** finds the leaf in the frame, and handles several leaves in one photo by classifying each region.
- **Classifier — 38 class scores from the ONNX model in about 0.7 ms.**
- **Advisory —** treatment name and dose, a local stock check, and the rupee value of the crop saved.
- **Offline demo —** a Gradio application that runs the whole path locally, with the network unplugged.

What the farmer sees

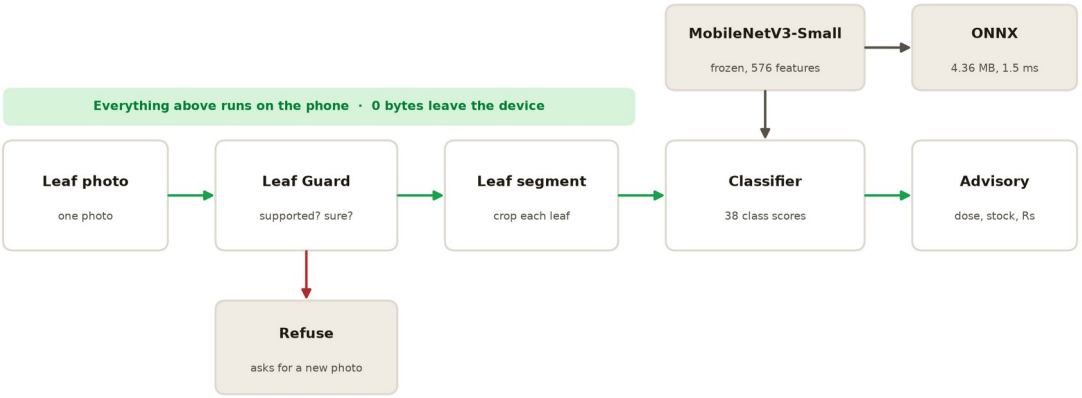
Three screens, one job each: take a photo, get a decision, or get an honest refusal with three tips on how to retake it. The design deliberately leads with the decision — dose, stock and rupees — rather than with a class name.



Interface design: capture, refusal, and the advisory. A refusal is a designed state, not an error.

5. How it works

The pipeline has one path and one real branch. An unsupported crop, or a photo the model is not sure about, exits at the guard before the classifier ever runs — so the app can say "not sure" instead of guessing.



Pipeline. Every stage runs on the device; nothing is sent anywhere.

The backbone is ImageNet-pretrained MobileNetV3-Small, used frozen as a feature extractor. A 576-dimensional feature vector is computed for each image, and a small two-layer head maps those features to 38 classes. Only the head is trained.

That choice is the whole design. Because the backbone never changes, training runs in minutes on a laptop CPU with no GPU, and the exported model stays at 4.36 MB — small enough to sit inside an application on a low-end phone.

6. The compact-AI angle

Compactness here is not a side effect of a small dataset; it is the constraint the design was built around.

Measure	Value	What it means
Model file	4.36 MB	Sits inside the app; no download on first run
Inference	0.7 ms	One CPU core, no GPU, no network round-trip
Training	Minutes on a laptop	No GPU cluster, no cloud training bill
Network calls at runtime	0	Works with the SIM out and the wifi off
Data leaving the device	0 bytes	Nothing to protect under DPDPA 2023

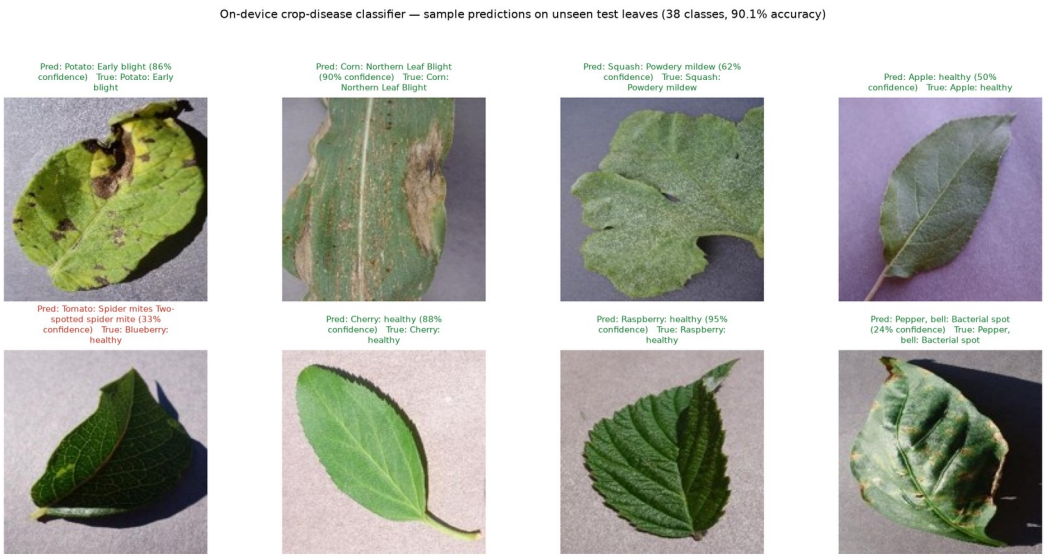
We also tried int8 dynamic quantization to shrink the model further. It cut the file size but collapsed validation accuracy to 2.8%, so we rejected it and shipped the fp32 model. Reporting that failure is more useful than hiding it: the small model we ship is the one that still works.

7. Results

All figures below are measured on the shipped model, on held-out PlantVillage images, on a 2-core CPU.

Figure	Value	Basis
Validation accuracy	90.1%	1,140 held-out images, 38 classes
Model size	4.36 MB	Exported ONNX, fp32

Figure	Value	Basis
Latency	0.7 ms / image	Single CPU core, 96×96 input
Classes	38	12 healthy + 26 disease or pest
Crops	14	Apple, blueberry, cherry, corn, grape, orange, peach, bell pepper, potato, raspberry, soybean, squash, strawberry, tomato



Sample predictions on unseen leaves. Correct answers in green, wrong answers in red — we show both.

8. Honest limitations

These are measured, not guessed. We would rather a judge know them from us than discover them later.

Limitation	The number	What it means for a farmer
Trained on lab images	**13–15% on real field images (PlantDoc benchmark)**	On a photo taken in a field, the model is far less reliable than the 90.1% headline suggests
Our own field test	**12% confidence on a real tomato leaf**	The guard refuses it, which is honest but not yet useful
Unsupported or sibling crops	**97% confident and wrong on a watermelon leaf; tomato vs potato late blight easily confused**	The app names the sibling crop and notes the treatment is identical. A supported-crop gate is the planned fix
Coverage	14 crops, 38 classes	Wheat, rice, sugarcane, cotton and pulses are missing — most of India's fields
Language	English only	A farmer who cannot read English cannot use it yet
Field trial	None	Nobody has measured this across a real season

We treat these as workstreams, not disclaimers. The next section is what we would do about each one.

9. What we would do next

Step	Why	Effort
Collect field photos and retrain	The single largest accuracy gain available to us	3 days
Add a supported-crop gate	Stops the confident wrong answer on unknown crops	2 days
Ship Hindi and a spoken answer	Reaches the farmer the app is actually for	4 days
Run one KVK pilot	Fifty farmers, one season, one measured number	1 season
Widen coverage with licensed data	PlantDoc and CC BY 4.0 detection sets, not unlicensed models	2 weeks

10. Tech stack and how to run it

Layer	Choice	Why this one
Model	MobileNetV3-Small (torchvision, ImageNet weights)	Small, fast on CPU, strong transfer features
Training	Frozen backbone + linear probe (PyTorch)	Minutes on a laptop, no GPU
Runtime	ONNX Runtime	Portable, no PyTorch needed at inference
Interface	Gradio	Fastest path to a working local demo
Image handling	Pillow, NumPy	Standard, no heavy dependencies

Reproduce it

```
py -3.12 -m venv venv  &&  .\venv\Scripts\activate
pip install -r requirements.txt
python app.py        # opens the local demo, no network needed
```

The repository also ships a complete training kit: a data preparation script, an augmented trainer with per-class metrics, a dataset checker, and a guide that takes a beginner from an empty laptop to a new model.

11. Risks and mitigation

Risk	Mitigation
Dataset bias — PlantVillage images are lab-quality, real field photos differ	Stated openly with measured numbers in Section 8; field collection is the next workstream
Quantization lowered accuracy	Tested int8, measured 2.8%, rejected it, shipped fp32

Risk	Mitigation
Unsupported crops get a confident answer	A supported-crop gate is specified and costed at two days
Time is short	Scope held to one working end-to-end path rather than a broad half-built one
Licence ambiguity in public datasets	Only permissive sources (CC0, CC BY 4.0, Apache 2.0) are used; unlicensed sets are excluded

12. Sources

Crop-loss figures in Section 3:

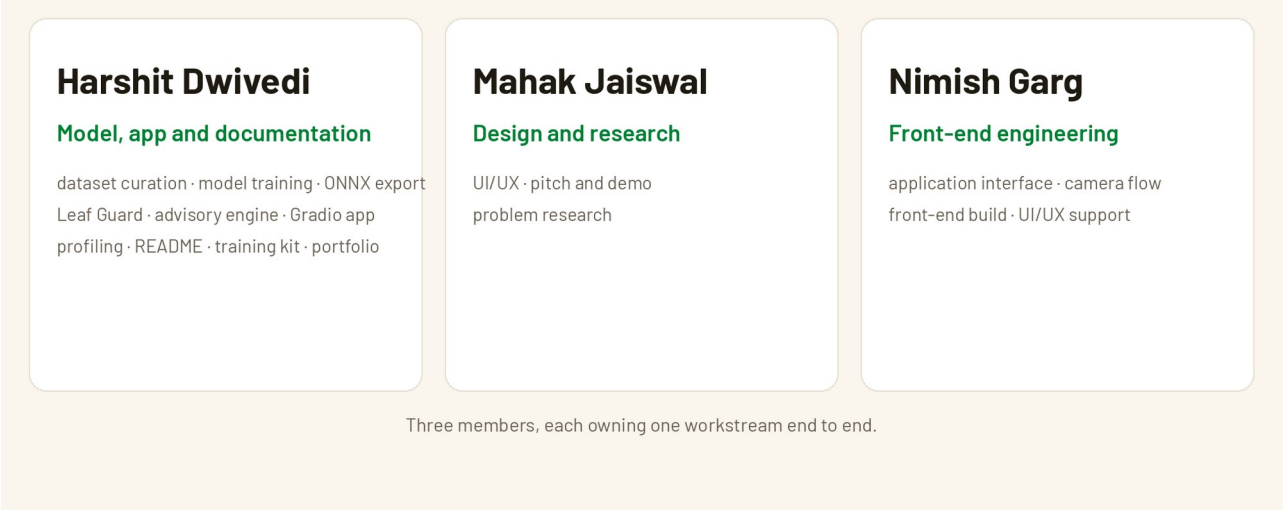
- Dhaliwal, G. S. et al. (2010). Pre- and post-harvest losses in agricultural produce — pest and disease loss estimates.
- National Academy Science Letters (2020). Crop loss estimation due to plant-parasitic nematodes in India.
- Tomato pest and disease management study for India and Bangladesh (2023).
- Agriculture Census of India (2015–16), small and marginal holdings share.

Model and data sources:

- Mohanty, S. P., Hughes, D. P. & Salathé, M. (2016). Using deep learning for image-based plant disease detection. Frontiers in Plant Science 7:1419. Dataset: github.com/spMohanty/PlantVillage-Dataset
- Singh, D., Jain, N., Jain, P., Kayal, P., Kumawat, S. & Batra, N. (2020). PlantDoc: A Dataset for Visual Plant Disease Detection. CoDS-COMAD 2020. The 13–15% field figure comes from this benchmark. Licence: CC BY 4.0.
- Howard, A. et al. (2019). Searching for MobileNetV3. Base weights via torchvision (BSD-3-Clause).

Licences of every library and dataset used are listed in the repository README.

13. Team and roles



The training kit in the repository means any of us can retrain the model without help.